Progetto di Ingegneria del Software 2025/26

Università Ca' Foscari Venezia

D5: Documento di Progettazione

Versione: 2.0

Gruppo: DSH

26/01/2026

Document Informations

AlertUp	AU
Deliverable	Documento di Progettazione
Data di Consegna	26/01/2026
Team Leader	Samuele Zanutto 890913@stud.unive.it
Team members	Davide Lago 895388@stud.unive.it Hamza Rofi 903405@stud.unive.it

Document History

Version	Issue Date	Stage	Changes	Contributors
1.0	30 / 11 / 2025	Draft	Definizione del Documento di Progettazione andando a definire l'architettura del sistema, il modello dei dati e del controllo, i modelli UML e la progettazione dell'interfaccia utente.	Samuele Zanutto, Davide Lago, Hamza Rofi.
2.0	26/01/2026	Final	Allineamento finale della documentazione: revisione generale del contenuto e correzione dettagli.	Samuele Zanutto, Davide Lago, Hamza Rofi.

Indice

1. Introduzione 4
1.1. Overview generale 4
1.2. Struttura del documento 4
2. Glossario 5
3. Architettura del Sistema 7
3.1. Modello e struttura del sistema 7
3.2. Gestione dei dati 8
4. Modello dei Dati e Controllo 9
4.1. Modello dei Dati 9
4.2. Modello di controllo 10
5. Modello UML 11
5.1. Diagramma delle classi 11
5.2. Diagramma delle attività 12
6. Progettazione dell'interfaccia Utente 14
6.1. Login 14
6.2. Registrazione 15
6.3. Home page 16
6.4 Dettaglio emergenza 17
6.4. Aggiungere emergenza 18
6.5. Aggiungere luogo preferito 19
6.6. Profilo 20
6.7. Notifiche 21
7. Riferimenti 21

1. Introduzione

1.1. Overview generale

Il presente documento costituisce il *Documento di Progettazione* dell'applicazione, un sistema mobile pensato per supportare i cittadini e i visitatori della città di Venezia attraverso la visualizzazione delle condizioni meteo-marine, delle emergenze presenti sul territorio e l'accesso rapido ai servizi di sicurezza e Protezione Civile.

La progettazione definisce nel dettaglio l'architettura software e hardware del sistema, illustrando la struttura generale dell'applicazione, il modello di comunicazione tra le sue componenti e la gestione dei dati attraverso un backend basato su **Node.js + MongoDB** e un frontend sviluppato in **Angular**, successivamente integrato in Android mediante WebView/packaging.

Il documento descrive inoltre:

- il **modello dei dati** e la **logica di controllo** adottata, evidenziando le principali entità, le loro relazioni e il flusso di gestione delle operazioni;
- il **modello architetturale** scelto (client–server e pattern Model-View-Controller per il frontend);
- i principali **diagrammi UML**, in particolare il diagramma delle classi e il diagramma delle attività relativo ai flussi più significativi;
- la **progettazione dell'interfaccia utente**, con mockup e descrizioni delle principali schermate: login, registrazione, homepage con mappa integrata, visualizzazione dettagli emergenza, aggiunta emergenza, profilo utente e sistema di notifiche.

L'obiettivo del documento è fornire una visione completa, coerente e dettagliata della struttura del sistema, utile sia per la fase di implementazione che per il futuro mantenimento e l'evoluzione dell'applicazione.

1.2. Struttura del documento

Il documento è articolato in sette capitoli principali:

1. **Introduzione** – Presenta lo scopo del documento e la sua struttura.
2. **Glossario** – Elenco dei termini chiave utilizzati nella progettazione del sistema.
3. **Architettura del Sistema** – Descrive il modello architetturale adottato, le componenti principali e la gestione dei dati.
4. **Modello dei Dati e del Controllo** – Approfondisce il modello dei dati (MongoDB) e il modello di controllo utilizzato dal sistema.
5. **Modello UML** – Include il diagramma delle classi e il diagramma delle attività relativi ai flussi principali.
6. **Progettazione dell'Interfaccia Utente** – Presenta e descrive tutte le schermate principali dell'applicazione.
7. **Riferimenti** – Riporta le fonti e il materiale di supporto utilizzato durante la progettazione.

Il documento segue il formato standard indicato dal corso e si integra con gli altri deliverable del progetto (D1, D2, D3 e D4), mantenendo una coerenza terminologica e metodologica con essi.

2. Glossario

API REST
Interfaccia di programmazione che consente la comunicazione tra frontend e backend tramite richieste HTTP standard (GET, POST, PUT, DELETE) e scambio di dati in formato JSON.
Fonte: Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures*
https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Angular
Framework frontend basato su TypeScript utilizzato per sviluppare applicazioni web single-page, impiegato nel progetto per la realizzazione dell'interfaccia client dell'app AlertUp.
Fonte: Documentazione ufficiale Angular
https://angular.io/docs

Backend
Componente server-side del sistema che gestisce la logica applicativa, l'accesso ai dati, l'autenticazione degli utenti e l'esposizione delle API REST.
Fonte: Sommerville, I. *Software Engineering*, Pearson

Call-Return Model
Modello di controllo centralizzato in cui un componente principale coordina l'esecuzione degli altri moduli tramite chiamate sequenziali che restituiscono il controllo al chiamante.
Fonte: Sommerville, I. *Software Engineering*, Pearson

Client–Server
Modello architetturale in cui i client richiedono servizi a un server centrale che elabora le richieste e restituisce le risposte.
Fonte: Tanenbaum, A. S., Van Steen, M. *Distributed Systems*
https://www.distributed-systems.net

Firebase Cloud Messaging (FCM)
Servizio di Google utilizzato per l'invio di notifiche push ai dispositivi mobili, integrato nel sistema per la gestione delle notifiche di emergenza.
Fonte: Documentazione ufficiale Firebase
https://firebase.google.com/docs/cloud-messaging

JSON (JavaScript Object Notation)
Formato testuale leggero per lo scambio di dati strutturati tra sistemi, utilizzato per la comunicazione tra frontend e backend.
Fonte: ECMA-404 Standard
https://www.json.org/json-en.html

JWT (JSON Web Token)
Meccanismo di autenticazione basato su token firmati digitalmente, utilizzato per verificare

l'identità dell'utente nelle comunicazioni client-server.
Fonte: RFC 7519
https://datatracker.ietf.org/doc/html/rfc7519

MongoDB
Database NoSQL orientato ai documenti utilizzato come repository centrale per la persistenza dei dati dell'applicazione.
Fonte: Documentazione ufficiale MongoDB
https://www.mongodb.com/docs

Model-View-Controller (MVC)
Pattern architetturale che separa la gestione dei dati (Model), la presentazione (View) e la logica di controllo (Controller), adottato nel frontend dell'applicazione.
Fonte: Gamma et al., *Design Patterns*
https://refactoring.guru/design-patterns/mvc

Node.js
Ambiente di runtime JavaScript server-side utilizzato per implementare il backend dell'applicazione.
Fonte: Documentazione ufficiale Node.js
https://nodejs.org/en/docs

Repository
Struttura dati centralizzata che consente ai diversi sottosistemi di accedere e modificare informazioni condivise, garantendo coerenza e indipendenza tra i moduli.
Fonte: Sommerville, I. *Software Engineering*, Pearson

Single Page Application (SPA)
Applicazione web che carica una singola pagina HTML e aggiorna dinamicamente il contenuto senza ricaricare l'intera pagina, paradigma adottato dal frontend Angular.
Fonte: MDN Web Docs
https://developer.mozilla.org/en-US/docs/Glossary/SPA

TTL (Time To Live)
Meccanismo che consente di eliminare automaticamente i dati dal database dopo un intervallo di tempo prefissato; nel progetto è usato per le emergenze con durata di 24 ore.
Fonte: MongoDB TTL Indexes
https://www.mongodb.com/docs/manual/core/index-ttl

WebView
Componente Android che permette di eseguire contenuti web all'interno di un'app mobile nativa, utilizzato per il packaging dell'app Angular.
Fonte: Android Developers Documentation
https://developer.android.com/reference/android/webkit/WebView.

3. Architettura del Sistema

3.1. Modello e struttura del sistema

L'architettura dell'applicazione segue un modello **client–server** suddiviso in più sottosistemi che cooperano per fornire servizi in tempo reale agli utenti. Il sistema è composto da:

Sottosistemi principali

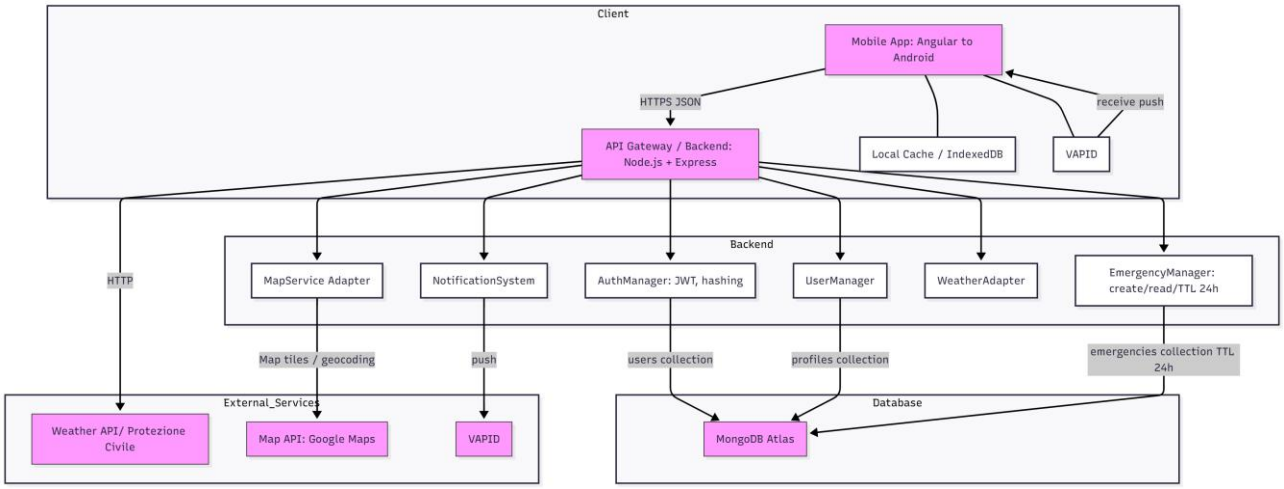
- **Frontend (Client Mobile)**
  Realizzato in **Angular**, successivamente integrato in Android tramite WebView/packaging.
  Si occupa dell'interazione con l'utente, della visualizzazione della mappa con le emergenze, delle notifiche locali, della consultazione meteo e della gestione del profilo.
- **Backend (Server REST API)**
  Implementato in **Node.js + Express**, espone servizi REST per:
  - autenticazione degli utenti,
  - memorizzazione e lettura dei dati utente,
  - gestione delle emergenze,
  - recupero informazioni meteo,
  - notifica verso client.
- **Database NoSQL (MongoDB)**
  Utilizzato per conservare i dati persistenti dell'applicazione:
  - dati utente (username, email, password hashed, città preferite),
  - storico minimizzato delle emergenze (emergenze con scadenza automatica a 24h tramite meccanismo TTL).
- **Servizi Esterni**
  L'applicazione integra servizi esterni quali:
  - API meteo
  - Collegamento al sito della Protezione Civile,
  - API per notifiche push (Firebase Cloud Messaging),
  - servizi di geolocalizzazione del dispositivo.

Struttura e flussi principali

Il sistema può essere rappresentato tramite una struttura a blocchi composta dai seguenti moduli:

- **AuthManager**: gestisce registrazione, login, token e hashing password.
- **UserManager**: gestisce dati profilo, città preferite e preferenze.
- **EmergencyManager**: crea, elimina e distribuisce emergenze; comunica con MongoDB e programma la rimozione automatica dopo 24h.
- **MapSystem**: gestisce la visualizzazione grafica della mappa e delle emergenze.
- **NotificationSystem**: riceve e genera notifiche push e notifiche locali.
- **WeatherSystem**: interroga i servizi meteo esterni e mostra le informazioni nell'app.
- **DataManager (backend)**: gestisce tutte le interazioni tra frontend, backend e database.

Il frontend comunica con il backend esclusivamente tramite **API REST in formato JSON**, garantendo una separazione netta tra logica di presentazione e logica applicativa.



3.2. Gestione dei dati

La gestione dei dati è suddivisa in diversi livelli, a seconda della natura dell'informazione e delle necessità di persistenza.

● Dati Persistenti: MongoDB

MongoDB rappresenta il database principale del sistema. È stato scelto per i seguenti motivi:

- modello **flessibile** e facilmente adattabile a dati eterogenei come le emergenze;
- performance elevate sotto carico e ottimo supporto per scritture frequenti;
- sistema integrato di **TTL (Time To Live)** che permette di eliminare automaticamente le emergenze dopo 24 ore senza dover implementare pulizia manuale;
- integrazione rapida con Node.js tramite libreria Mongoose.

Dati gestiti nel database

- **Utenti**:
  - username
  - email
  - password (hash + salt)
  - città preferite
  - impostazioni notifiche
- **Emergenze (TTL 24h)**:
  - titolo
  - descrizione
  - posizione GPS
  - tipo emergenza
  - timestamp
  - data di scadenza automatica

Gli utenti possono modificare i propri dati, mentre le emergenze non possono essere modificate dopo la creazione (solo rimosse automaticamente).

● Dati temporanei o locali nel client

Il sistema utilizza una piccola porzione di memoria locale (Local Storage o IndexedDB) per migliorare l'esperienza utente:

- token di autenticazione (validità limitata),
- stato UI (es. ultima schermata aperta),

Non vengono memorizzati in locale dati sensibili permanenti.

● **Comunicazione tra app e server**

La comunicazione avviene tramite:

- **API REST** su protocollo HTTPS,
- scambio dati in formato **JSON**,
- autenticazione tramite **JWT**,
- notifiche via **Firebase Cloud Messaging**

Questo approccio garantisce:

- elevata sicurezza,
- interoperabilità,
- integrazione rapida con strumenti di debugging.

4. Modello dei Dati e Controllo

4.1. Modello dei Dati

Il modello dei dati adottato per il sistema è un **modello a repository**, nel quale tutti i sottosistemi accedono e modificano un'unica struttura persistente condivisa: **MongoDB Atlas**.
La repository centralizza la gestione dei dati fondamentali del sistema, rendendola indipendente rispetto ai componenti software che la utilizzano.

Struttura generale del repository

Il database è costituito da tre collezioni principali:

- **Users**
  - username
  - email
  - password (hashed)
  - città preferite
  - impostazioni utente
- **Emergencies**
  - tipo emergenza
  - descrizione
  - posizione geografica
- **Notifications**
  - userId
  - contenuto notifica
  - timestamp
  - stato lettura

Interazione con il repository

I vari sottosistemi dell'app (AuthManager, EmergencyManager, NotificationSystem, UserManager) non comunicano tra loro direttamente, ma scambiano dati **solo attraverso la repository**. Questa scelta:

- semplifica l'indipendenza tra i moduli;

- facilita il tracciamento delle modifiche;
- garantisce coerenza dei dati centralizzando ogni operazione CRUD.

Dati locali

Una minima quantità di dati è gestita anche in locale tramite:
- **Local Storage / IndexedDB** per caching temporaneo (token, dati utente),
- **Cache volatile dell'app** per migliorare l'esperienza offline limitata.

Tuttavia, la repository principale rimane **MongoDB**, responsabile della persistenza e della consistenza dei dati.

4.2. Modello di controllo

Il sistema adotta un **modello di controllo centralizzato**, più precisamente la variante **call-return**, nella quale un unico componente funge da nodo di coordinamento dell'intera applicazione lato backend: il **Backend Controller** (Node.js + Express).

Struttura del controllo

Il controllo è strutturato gerarchicamente:
1. **Backend Controller (root del controllo)**
    - Gestisce le richieste provenienti dall'app mobile.
    - Valida gli input e seleziona il modulo di business appropriato.
2. **Sottosistemi specializzati** (livelli inferiori)
    - **AuthManager**: gestione registrazione/login/token.
    - **EmergencyManager**: gestione emergenze e eliminazione automatica tramite TTL del DB.
    - **UserManager**: profili, preferenze e dati personali.
    - **NotificationManager**: invio notifiche tramite **Firebase Cloud Messaging**.

    - **Adapter esterni**: mappe, meteo, Protezione Civile.

Ogni sottosistema è invocato tramite una **chiamata sequenziale** (call-return) che parte sempre dall'entità centrale.

Motivazioni della scelta

L'adozione di un modello di controllo centralizzato call-return consente di:
- mantenere un controllo chiaro e univoco del flusso applicativo,
- semplificare la gestione delle chiamate al database,
- garantire maggiore sicurezza grazie a un unico punto di validazione delle richieste,
- facilitare la manutenzione e il debugging,
- assicurare che tutte le funzionalità rispettino policy e vincoli non funzionali (es. sicurezza, tempi di risposta).

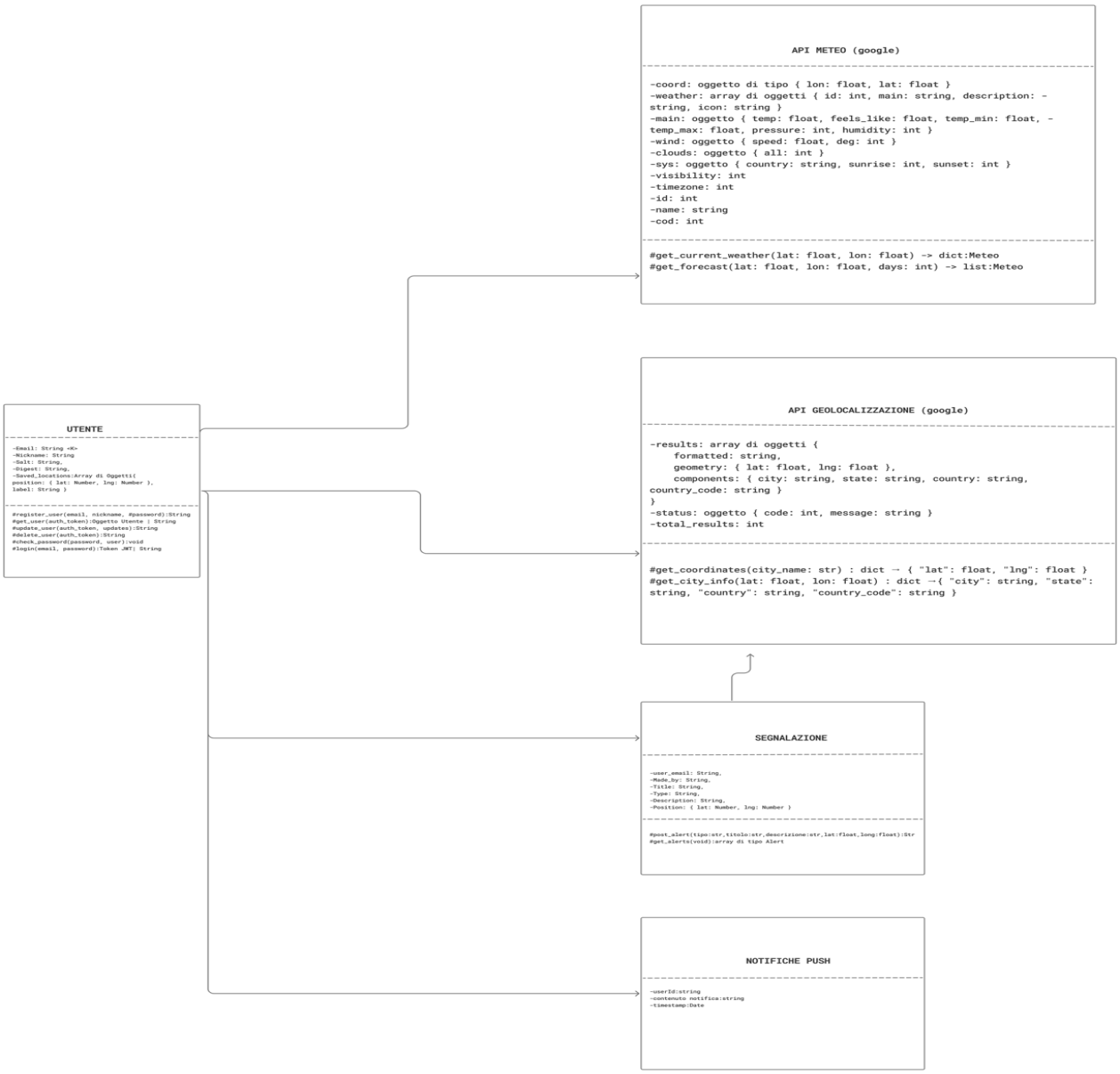
Comportamento generale
- Il client → invia una richiesta al Backend Controller.
- Il controller identifica il servizio richiesto.
- Chiama il modulo corretto.
- Il modulo interagisce con MongoDB.
- Il risultato ritorna al controller → che risponde al client.

Questa pipeline garantisce un flusso ordinato, prevedibile e facilmente estendibile.

5. Modello UML

5.1. Diagramma delle classi

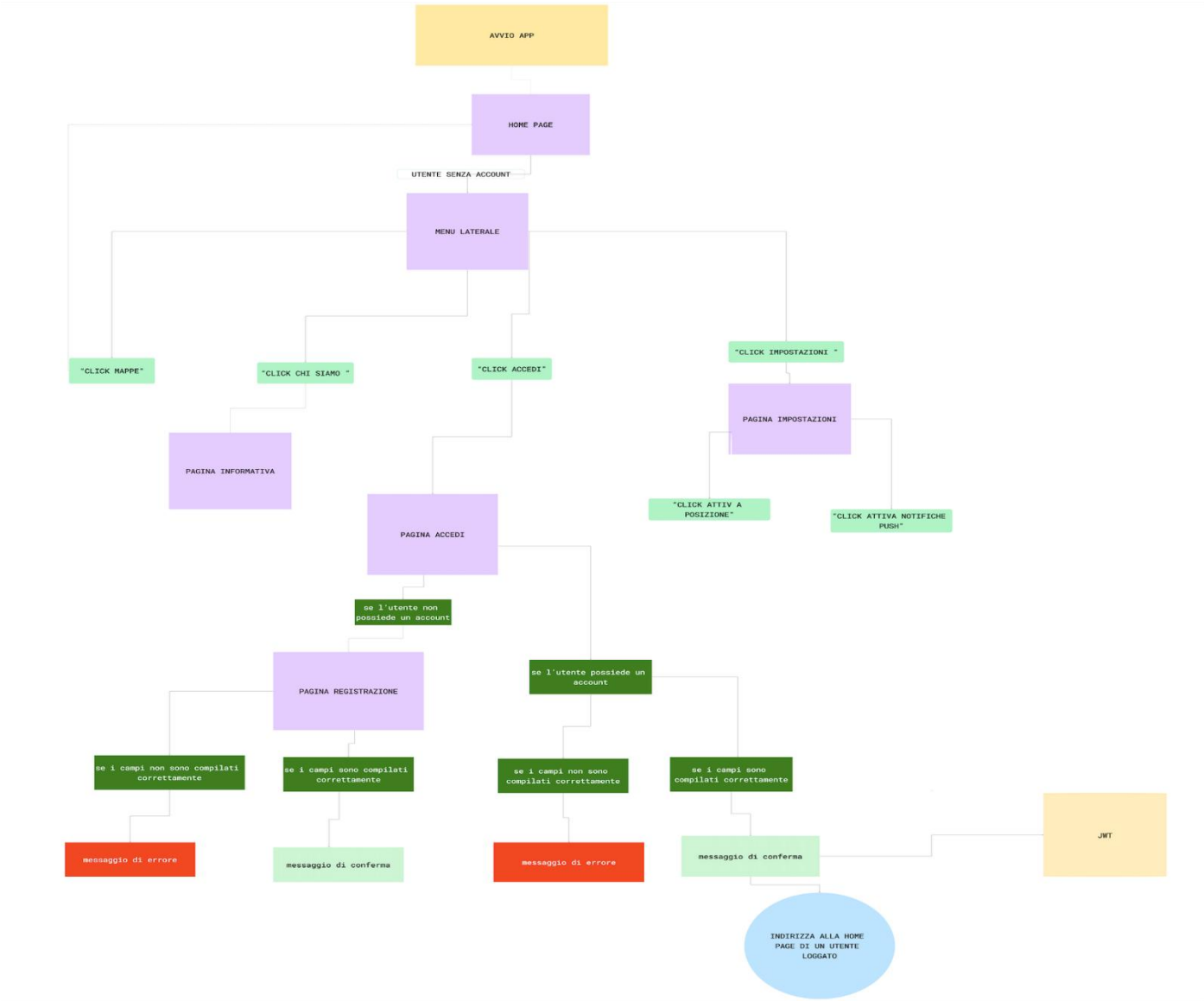
Il seguente diagramma delle classi rappresenta la struttura logica dell'app AlertUp, evidenziando le principali entità del sistema, i loro attributi, i metodi e le relazioni che ne definiscono il comportamento.



5.2. Diagramma delle attività

Il seguente diagramma delle attività illustra il flusso operativo dell'app AlertUp, mostrando passo dopo passo come l'utente interagisce con il sistema e come l'app gestisce le notifiche di emergenza.

Diagramma delle attività di un utente non loggato.

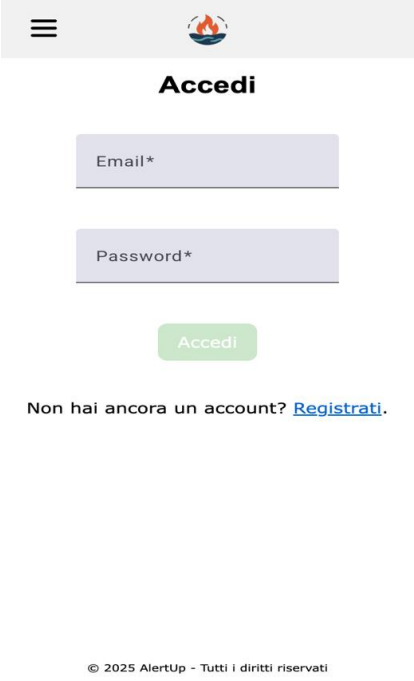


AVVIO APP
HOME PAGE
UTENTE CON ACCOUNT
MENU LATERALE
"CLICK MAPPE"
"CLICK CHI SIAMO "
"CLICK SEGNALA EMERGENZE"
"CLICK IMPOSTAZIONI "
"CLICK ESCI "
PAGINA INFORMATIVA
PAGINA IMPOSTAZIONE
INDIRIZZA ALLA HOME PAGE DI UN UTENTE NON LOGGATO
"CLICK ATTIVA POSIZIONE"
"CLICK ATTIVA NOTIFICHE PUSH"
PAGINA EMERGENZE
PAGINA IMPOSTAZIONE ACCOUNT
"CLICK CAMBIA PASSWORD"
"CLICK CAMBIA NICKNAME"
"CLICK CAMBIA EMAIL"
"CLICK ELIMINA ACCOUNT"
se i campi non sono compilati correttamente
se i campi sono compilati correttamente
messaggio di errore
messaggio di conferma
SEGNALAZIONE VISUALIZZATA NELLA HOME PAGE
se i campi non sono compilati correttamente
se i campi sono compilati correttamente
messaggio di errore
messaggio di conferma
se i campi non sono compilati correttamente
se i campi sono compilati correttamente
messaggio di errore
messaggio di conferma
se i campi non sono compilati correttamente
se i campi sono compilati correttamente
messaggio di errore
messaggio di conferma
se l'utente non accetta
se l'utente accetta
messaggio di conferma
INDIRIZZA ALLA HOME PAGE DI UN UTENTE NON LOGGATO

6. Progettazione dell’interfaccia Utente

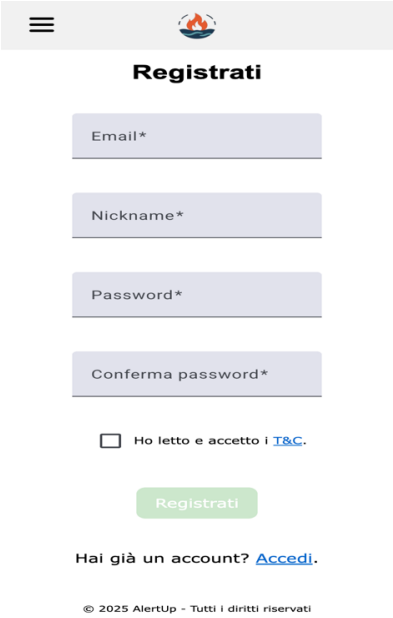
6.1. Login

La schermata di login permette l’autenticazione dell’utente tramite credenziali verificate dal backend, abilitando l’accesso personalizzato alle funzionalità dell’app e assicurando la protezione dei dati attraverso protocolli di sicurezza



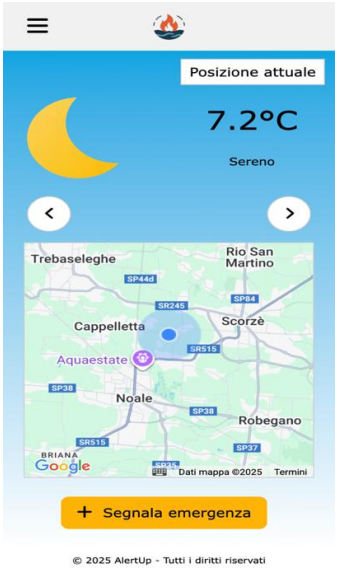
6.2. Registrazione

La schermata di registrazione consente all'utente di creare un nuovo account all'interno del sistema AlertUp, raccogliendo e validando le informazioni necessarie tramite controlli lato client e lato server, per garantire l'integrità dei dati e l'accesso sicuro ai servizi dell'app.



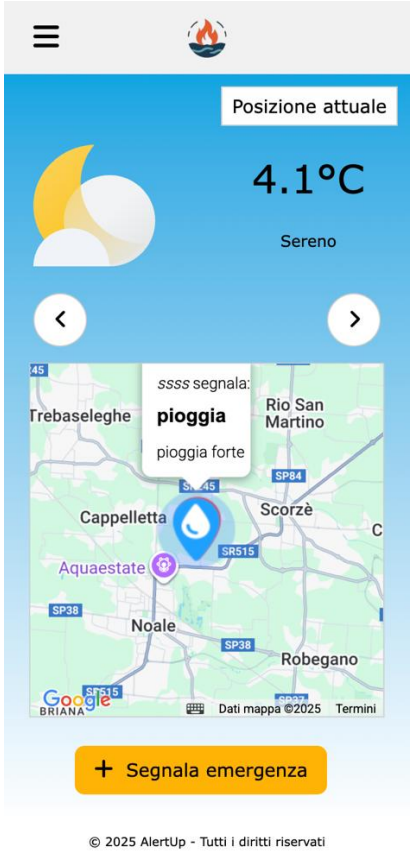
6.3. Home page

La Home Page funge da dashboard principale dell'app AlertUp, aggregando le funzioni essenziali e offrendo un accesso rapido alle principali sezioni del sistema mediante un'interfaccia chiara e ottimizzata per la navigazione e per la visualizzazione delle emergenze.



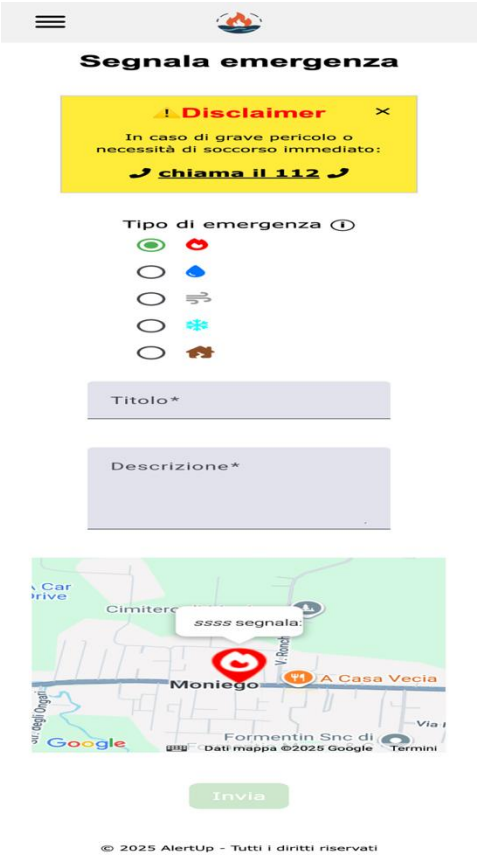
6.4 Dettaglio emergenza

La schermata dei dettagli emergenza presenta una visualizzazione strutturata delle informazioni relative alle emergenze dei luoghi preferiti e la localizzazione attuale.



6.4. Aggiungere emergenza

La schermata 'Aggiungi emergenza' consente all’utente di registrare un nuovo evento critico, inserendo i dati necessari e inviandoli al sistema per la memorizzazione e l’attivazione delle azioni previste dal flusso di gestione delle emergenze."

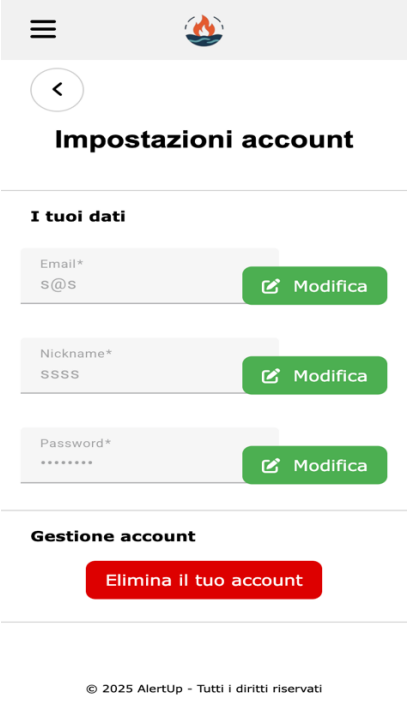


6.5. Aggiungere luogo preferito

La schermata 'Aggiungi luogo preferito' permette all’utente di salvare un nuovo punto di interesse, acquisendo i dati relativi alla posizione e integrandoli nel profilo utente per una gestione personalizzata delle notifiche e delle segnalazioni.

6.6. Profilo

La schermata profilo mostra e gestisce le informazioni personali dell'utente, consentendo la modifica dei dati e la sincronizzazione con il database centrale per mantenere aggiornate le preferenze e i parametri associati all'account.



6.7. Notifiche

Questa schermata mostra una notifica push inviata dall'app AlertUp.

7. Riferimenti

Le risorse utilizzate per la stesura di questo documento sono:

- Slides e risorse scaricate dalla piattaforma moodle del corso.
- Documentazione ufficiale Angular (https://angular.dev).
- Documentazione **Node.js** (https://nodejs.org/en/docs).
- Manuale tecnico **MongoDB Atlas** (https://www.mongodb.com/docs/atlas).